Reconstructing Missing Indoor CO2 Sensor Readings

MMA3001 Dataset Project – Dataset 2
Name: Maeann Sim Chiew Hsia
Student ID: 33606102
GitHub repository: https://github.com/msim0039-bit/mma3001-sensor-project

1. Engineering problem

Indoor CO_2 monitoring can support ventilation assessment in buildings. However, CO_2 readings alone do not establish that an indoor environment is safe (Health and Safety Executive [HSE], n.d.). Reconstructing these readings could help with retrospective analysis, but inaccurate estimates may hide changes or introduce artificial peaks.

This project asks: How accurately can missing indoor CO_2 readings be reconstructed using linear interpolation, natural cubic spline interpolation and multiple linear regression, and how does reconstruction error change with gap length?

The investigation uses environmental measurements from Dataset 2 for sensor 6012002000869. Known CO_2 readings are temporarily hidden in groups of 6, 18 and 36 consecutive readings. The three methods are compared against a previous-value baseline using mean absolute error (MAE) and root mean square error (RMSE), measured in parts per million (ppm).

The intended use is offline reconstruction of historical records, where observations after a gap are available. The study covers one sensor and simulated missing-data events; it does not establish sensor accuracy, calculate ventilation rates or evaluate real-time control.

2. Inputs, outputs and data

2.1 Inputs

The input is 5EnvSensor_MayToDec2024_180kRows.zip from the unit-supplied Dataset 2 folder “MMA3001 Dataset - MSI Occupancy and Environmental” (n.d.). It contains CSV records with sensor identifiers and JSON-encoded measurements. The analysis selects sensor 6012002000869.

Table 1. Input variables and their uses

Variable	**Unit**	Purpose
Carbon dioxide	ppm	Observed values used for training, interpolation and evaluation
Temperature	°C	Regression predictor
Humidity	%	Regression predictor
Pressure	mb	Regression predictor
Timestamp	Unix seconds	Ordering, alignment and elapsed-time calculations

The audit found 29,341 CSV rows for this sensor but 13,800 distinct CO_2 timestamps. Row count therefore does not represent the number of unique observations.

The loader in src/compare.py collapses identical timestamp-value pairs, aligns the four variables by exact timestamp and sorts them chronologically. The first 50% trains regression, the next 25% supports development comparisons, and the final 25% provides a later-period robustness check. The later period had already been examined, so it is not treated as an untouched final test set.

The numerical methods require finite values and increasing observation timestamps. Interpolation targets must fall within the supplied observation range. No physical concentration limits are enforced, so these checks do not establish whether a measurement is physically plausible.

2.2 Outputs

The main numerical output is an estimated CO_2 concentration in ppm for each hidden timestamp. Estimates are compared with the recorded values to calculate MAE and RMSE; lower values indicate closer agreement with those records.

Saved CSV tables report errors, gap counts and elapsed boundary intervals. Additional outputs record runtime, spline-context sensitivity, later-period performance and individual case predictions.

Gap sizes are reported as 6, 18 and 36 hidden readings. Their elapsed durations depend on the actual timestamps. The 24-hour case windows describe the viewing period, rather than the duration of the missing interval.

2.3 Invalid or Unsupported Inputs

The main loader stops with an error if required columns or variables are missing, measurements are invalid or non-finite, different values occur at the same timestamp, or the four variables are incompletely aligned. These problems are reported rather than silently filled or averaged.

The reconstruction functions also check relevant conditions, including timestamp order, interpolation range and regression training requirements.

3. Computational Approach

3.1 Workflow

The analysis was carried out in Python through Google Colab. GitHub stores the source modules, notebook, tests, documentation and recorded results.

Four methods reconstruct the same hidden readings: a previous-value baseline, linear interpolation, natural cubic spline interpolation and multiple linear regression. Hidden CO_2 values are excluded from reconstruction inputs and used only to calculate errors.

3.2 Previous-Value Baseline

The previous-value method repeats the last observed CO_2 concentration before the gap:

$$\acute{C}(t) = C_L$$

Here, C_L is the concentration immediately before the gap, and $\acute{C}(t)$ is the estimated concentration at a missing timestamp. Both are expressed in ppm.

This method assumes that concentration remains constant throughout the gap. It provides a simple baseline for checking whether the other methods improve accuracy.

3.3 Linear Interpolation

Linear interpolation connects the observations immediately before and after a gap:

$$\acute{C}(t) = C_L + (C_R - C_L)\frac{t - t_L}{t_R - t_L}$$

Here, t_L and t_R are the boundary timestamps, and C_L and C_R are their observed concentrations. Each target timestamp satisfies $t_L < t < t_R$.

The calculation uses elapsed time rather than reading position, which accommodates uneven measurement intervals.

Linear interpolation was included because it is simple, interpretable and requires only two boundary observations. It assumes a constant rate of concentration change across the gap. Estimates remain between the boundary concentrations, but the method cannot recover an unobserved peak or dip.

The implementation in src/reconstruct.py rejects equal or reversed boundary timestamps, targets outside the gap and unsupported method names. It returns one estimate per target timestamp in the supplied order.

3.4 Natural Cubic Spline Interpolation

Cubic spline interpolation fits a cubic polynomial between neighbouring observed timestamps:

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3$$

Here, t_i is the start of interval i, and the coefficients define the curve on that interval. Neighbouring polynomials join with continuous first and second derivatives.

The implementation uses SciPy's CubicSpline with natural boundary conditions. These set the second derivative to zero at the first and last supplied observations (SciPy, n.d.). The main comparison supplies four observed CO_2 readings on each side of a gap. Hidden readings are excluded, and timestamps are converted to seconds relative to the first context observation.

Spline interpolation was included to investigate whether a curved estimate could follow CO_2 changes better than a straight line. Splines can overshoot between observations, particularly across longer gaps. Extrapolation is disabled, and predictions are not clipped, so overshoot remains visible.

The function in src/models.py checks input shapes, finite values, increasing observation timestamps and queries within the observed range. A separate sensitivity experiment compares two, four and eight context observations per side.

3.5 Multiple Linear Regression

Multiple linear regression estimates CO_2 concentration from temperature, humidity and pressure:

$$\acute{C} = \beta_0 + \beta_T z_T + \beta_H z_H + \beta_P z_P$$

Here, β_0 is the intercept, and the remaining coefficients describe the fitted contributions of the standardized predictors. Each predictor is standardized using:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

The mean μ_j and standard deviation σ_j are calculated from the first 50% of chronological observations. NumPy's linalg.lstsq fits the coefficients by minimizing the sum of squared training residuals (NumPy, n.d.). Predictions use the stored training scaling and coefficients without refitting.

Unlike interpolation, it does not directly use the CO_2 observations surrounding each gap. Temperature, humidity and pressure must remain available at the target timestamps, so this represents loss of the CO_2 channel rather than failure of the entire sensor.

The model assumes a linear relationship between the predictors and CO_2. It describes associations rather than causal effects, and performance may decrease when conditions differ from the training period. The implementation rejects non-finite inputs, constant training features and a rank-deficient training design matrix.

3.6 Settings and basis for comparison

In the main comparison, each gap and its four-point spline context must remain within the evaluation period. Windows are skipped if any adjacent observation interval in that context exceeds 30 minutes. This screening rule is not a validated maximum gap duration.

MAE and RMSE assess agreement with the hidden readings. Runtime, spline-context sensitivity, later-period performance and additional daily cases provide further evidence for choosing a method.

4. Validation method and results

4.1 Checking the implementation

Verification checked whether the functions produced expected numerical answers. The repository contains 13 unit tests covering the previous-value baseline, linear interpolation, cubic spline interpolation and regression. Examples include reconstructing a straight line at uneven timestamps, checking that the spline passes through supplied observations and predicting values from a known regression relationship. Selected invalid inputs were also tested, including targets outside a gap, non-finite values, constant regression features and rank-deficient training data.

All 13 unit tests passed in the recorded run. The executable documentation examples also passed.

A separate reproduction check ran src/compare.py from a fresh repository copy in Google Colab using the original environmental ZIP. The gap counts, MAE, RMSE and median boundary intervals matched the saved comparison within an absolute tolerance of $10^{-6,}$ with relative tolerance set to zero. The commands and outputs are recorded in the notebook, TEST_REPORT.md and the evidence/ folder.

4.2 Evaluating reconstruction accuracy

Accuracy was evaluated by temporarily hiding consecutive groups of 6, 18 and 36 recorded CO_2 readings.

Regression used scaling and coefficients fitted only on the earlier training period. Interpolation and spline inputs excluded the hidden CO_2 values.

In the development comparison, candidate gap starts were separated by the gap size plus eight readings. Each gap and its spline context had to remain within the evaluation period. Windows with an adjacent timestamp interval above 30 minutes were skipped. This produced 246, 132 and 78 eligible gaps for the three sizes.

For each hidden reading, error was calculated as:

$$e_i = \acute{C}_i - C_i$$

where $\acute{C}_i$ is the estimate and C_i is the recorded concentration. Accuracy was measured using:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}\left|e_i\right|$$

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}$$

Here, N is the number of hidden readings evaluated. Both metrics are expressed in ppm. MAE describes average absolute error, while RMSE gives greater weight to large errors. Errors are pooled across all hidden readings for each method and gap size.

4.3 Development results

Table 2. Development reconstruction errors: MAE / RMSE (ppm)

Method	6 hidden readings	18 hidden readings	36 hidden readings
Previous value	13.05 / 18.16	17.04 / 23.49	30.75 / 41.68
Linear interpolation	9.81 / 12.82	11.62 / 15.22	16.60 / 21.88
Natural cubic spline	18.49 / 23.61	39.31 / 52.10	75.38 / 102.62
Multiple linear regression	35.19 / 45.93	35.47 / 46.16	35.62 / 46.34
Gaps evaluated	246	132	78

Linear interpolation had the lowest aggregate MAE and RMSE at all three gap sizes. Its MAE increased from 9.81 to 16.60 ppm as more readings were hidden.

The median elapsed intervals between the surrounding observations were approximately 80, 210 and 400 minutes.

4.4 Additional Daily Cases

Following TA feedback, ten 24-hour viewing windows were examined: five from the development period and five from the later period. Eligible windows were spread chronologically using a fixed selection procedure that did not use reconstruction errors.

Gaps were placed near hours 6, 12 and 18 relative to each window's start. Testing all three gap sizes at each position produced 90 simulated gaps and 360 method results. The 24-hour duration describes the viewing window, not the missing interval.

Checks confirmed that gaps and their context stayed within the selected window and evaluation period, hidden CO_2 values were excluded from spline inputs, and predictions had the expected shape and finite values. Recalculating MAE and RMSE from saved individual predictions matched the reported metrics within an absolute tolerance of 10^{-9}.

Linear interpolation had the uniquely lowest MAE in 60 cases, previous value in 24, regression in four and spline in one. One case had a tied minimum. Selected case figures are discussed in Section 5, and the complete tables are saved in results/co2_position_cases/.

4.5 What the validation establishes

The results support a relative method choice for the investigated sensor and simulated gaps. No application-specific acceptable error was established, so lower MAE and RMSE do not demonstrate suitability for ventilation control or accurate threshold detection.

The later-period results are treated as a robustness check because those data had already been examined. Some daily cases overlap and are not statistically independent. Agreement with hidden readings also does not establish the physical accuracy of the original sensor.

5. Comparison and optimisation work

5.1 Comparing the methods

The development results in Table 2 favour linear interpolation. Compared with the previous-value baseline, it reduced MAE by approximately 25%, 32% and 46% for gaps of 6, 18 and 36 readings.

The natural cubic spline produced larger errors, particularly across longer gaps. For these gaps, nearby CO_2 observations were more useful than the fitted relationship with temperature, humidity and pressure.

Linear interpolation performed best overall, but other methods were better in some cases. Gap position alone did not provide a reliable rule for choosing a method.

5.2 Behaviour in selected cases

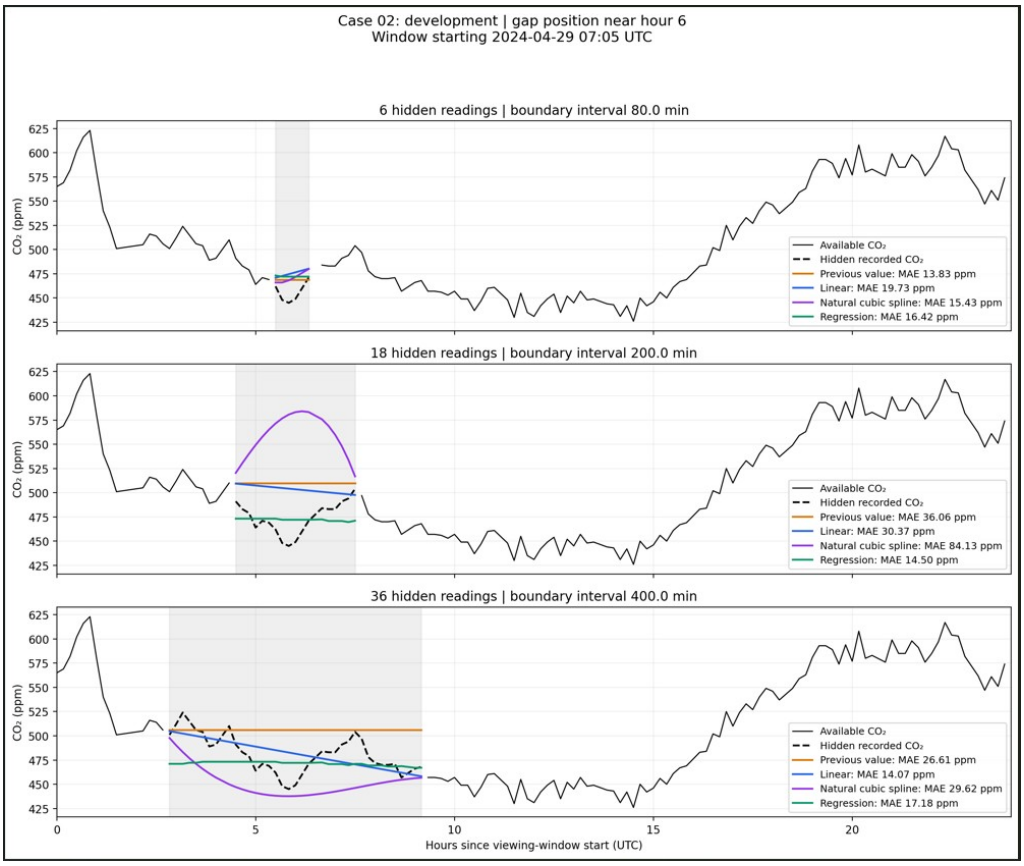


Figure 1. Reconstruction comparison for Case 2, with gaps near hour 6 of the viewing window. The panels show 6, 18 and 36 hidden readings.

In Case 2, the hidden concentrations included a dip below both boundary observations. For the 18-reading gap, regression achieved an MAE of 14.50 ppm, compared with 30.37 ppm for linear interpolation and 84.13 ppm for the spline. However, the preferred method changed with gap size: previous value had the lowest MAE for six readings, while linear interpolation performed best for 36.

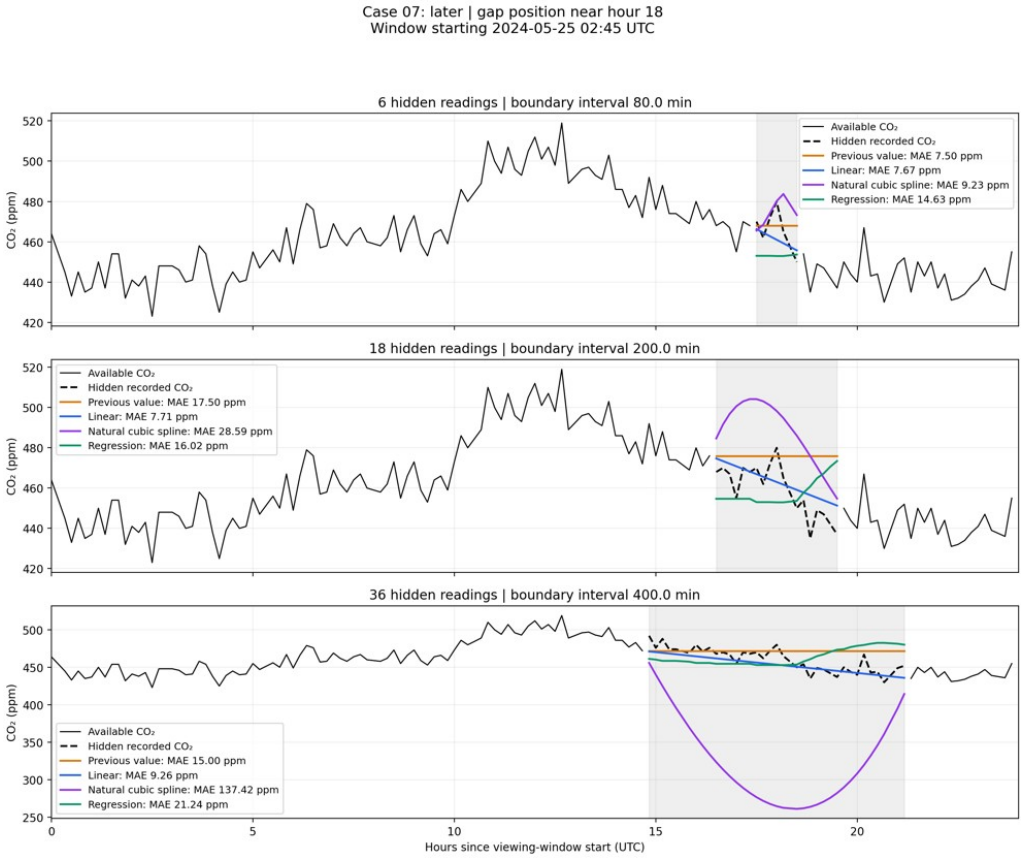


Figure 2. Reconstruction comparison for Case 7, with gaps near hour 18 of the viewing window. The panels show 6, 18 and 36 hidden readings.

Case 7 illustrates the risk of spline overshoot. For the 36-reading gap, spline MAE reached 137.42 ppm, compared with 9.26 ppm for linear interpolation. The previous-value baseline slightly outperformed linear interpolation for the six-reading gap, with MAEs of 7.50 and 7.67 ppm respectively.

These figures were selected after examining the results to illustrate different behaviours. They are not an unbiased sample of all gaps; the complete case tables provide the broader evidence.

5.3 Sensitivity to spline context

Spline context was varied to investigate whether supplying more surrounding observations improved accuracy. Two, four and eight observations per side were compared using the same eligible gaps.

Table 3. Effect of spline context on MAE (ppm)

Method	6 hidden readings	18 hidden readings	36 hidden readings
Linear interpolation	9.12	12.74	17.40
Spline: 2 observations per side	15.19	31.75	48.06
Spline: 4 observations per side	18.42	41.46	68.32

Method	6 hidden readings	18 hidden readings	36 hidden readings
Spline: 8 observations per side	18.46	41.59	68.73
Gaps evaluated	156	101	66

Two observations per side gave the lowest spline MAE at every gap size. For 36 hidden readings, this reduced MAE by approximately 30% compared with four observations per side. However, linear interpolation still had lower errors.

These gaps differ from those in the main comparison because eligibility required enough context for all settings. Comparisons should therefore be made within Table 3 rather than treating changes from Table 2 as effects of context alone.

5.4 Later-period robustness

The methods were also evaluated in the final 25% of chronological observations. Regression retained its original training scaling and coefficients.

Table 4. Later-period MAE (ppm)

Method	6 hidden readings	18 hidden readings	36 hidden readings
Previous value	10.17	13.94	15.93
Linear interpolation	8.32	9.47	11.69
Spline: 4 observations per side	15.99	36.13	63.86
Spline: 2 observations per side	13.32	26.52	48.22
Multiple linear regression	23.73	23.69	23.28
Gaps evaluated	246	132	78

Linear interpolation again had the lowest aggregate MAE and RMSE. Using two context observations improved the spline relative to four but did not change the overall method choice.

Errors were generally lower than in the development period. This reflects performance on different concentration patterns rather than evidence that the models improved over time.

5.5 Computational performance and method selection

Timing measurements were collected in Google Colab using repeated calls after warm-up. The recorded median times for reconstructing one selected gap were approximately 0.008 ms for previous value, 0.028 ms for linear interpolation, 0.480 ms for spline and 0.029 ms for regression. Regression training took approximately 0.932 ms and was measured separately.

The reconstruction timings exclude file loading and regression training; spline timing includes constructing the spline. These measurements describe one gap and one runtime environment, so they should not be treated as universal speed estimates. Full timing details are recorded in the repository evidence.

All methods were fast for the investigated task, so accuracy and data requirements were more useful selection criteria than these small timing differences. Linear interpolation combined the lowest aggregate errors with simple inputs and low computational cost.

6. Limitations and lessons learned

6.1 Engineering usefulness and limitations

Following the broad concentration trend does not guarantee that a method preserves short peaks, dips or the timing of changes.

The reported MAE and RMSE describe average agreement with recorded measurements, not bounds on individual errors. An estimate close to a decision threshold could lead to a different interpretation even when average error is small. This project therefore supports a relative method choice but does not establish an operational accuracy requirement.

Reconstructed values should remain clearly flagged, with the original measurements and missing-data locations preserved.

6.2 Limits of the investigation

The investigation uses one sensor and artificially hidden measurements. Real outages may be associated with sensor faults, communication failures or unusual environmental conditions.

Regression assumes temperature, humidity and pressure remain available during the CO_2 gap. If the entire sensor stops recording, those predictors would also be unavailable. Interpolation requires an observation after the gap, so its tested use is offline reconstruction rather than immediate ventilation control.

The later period had already been examined, and some daily cases overlap. These comparisons provide useful supporting evidence but are not independent final validation.

Finally, numerical correctness and physical accuracy are different. Passing tests supports the tested calculations, while matching hidden readings supports agreement with those records. Neither verifies sensor calibration or removes bias in the original measurements.

6.3 Lessons learned and further work

A main lesson was that greater complexity did not necessarily improve reconstruction. Regression offered a different source of information, but the selected predictors were less effective than nearby CO_2 observations for most investigated gaps.

The additional daily cases showed why one favourable graph is insufficient for choosing a method.

Further work should test additional sensors and an unexamined chronological period, with method settings fixed beforehand. Evaluation should also consider realistic outage patterns, maximum errors, peak preservation and threshold-crossing behaviour. An acceptable error should be defined for the intended application before practical use.

For the current task, linear interpolation is a reasonable starting method when valid boundary observations are available. The reconstruction method, boundary timestamps and elapsed gap duration should be recorded so that estimates can be reviewed and distinguished from measured data.

7. AI-use reflection

7.1 Tools and contribution

I used ChatGPT to brainstorm the investigation, explain numerical methods, assist with Python code, suggest tests and improve documentation. It also helped draft and revise parts of

the report. I used Gemini in Google Colab to explain errors, troubleshoot code and develop plotting code.

My work involved running the experiments, checking outputs, incorporating TA feedback and organising the code, results and report. AI suggestions provided a starting point, while the recorded results formed the basis for the method comparison and conclusions.

7.2 Checking AI-assisted work

I checked the numerical implementations through the recorded unit tests and executable documentation examples. A separate reproduction check compared the source-code results with the saved comparison table.

Data checks were also important. Repeated measurements meant that CSV row count was different from the number of unique observations. The audit and stricter loader made this distinction visible and checked for conflicting measurements and incomplete alignment.

For the reconstruction experiments, hidden CO_2 readings were excluded from model inputs, and regression scaling and coefficients came only from the training period. Recalculating errors from saved predictions provided another consistency check.

Some checking code was also AI-assisted, so passing these checks does not prove that every possible error was found. Tests based on known answers and comparisons between saved predictions and reported metrics provided stronger evidence than accepting generated code without running it.

7.3 Limitations of the assistance

AI sometimes suggested changes in small fragments, which made it difficult to track where code or text belonged. Complete code blocks and checks of the saved outputs were more useful than repeatedly applying isolated edits.

I also used ChatGPT to review the report against the rubric and identify areas needing clearer explanation or supporting evidence. I checked these suggestions against the project brief before making changes.

AI could help describe a smooth spline curve, but the results showed that smoothness did not imply better accuracy. These points required checking the reasoning behind the conclusions, rather than only checking whether the code ran.

7.4 What I learned

The project helped me distinguish implementation verification from reconstruction accuracy and physical sensor validity. I also learned why training information must be separated from evaluation targets, why gap duration should use actual timestamps, and why individual cases matter alongside average errors.

In future work, I would fix the evaluation plan earlier, reserve an unexamined period and develop the workflow in smaller stages. I remain responsible for the submitted work and for explaining how the methods and evaluation operate.

8. References

Health and Safety Executive. (n.d.). *Using CO_2 monitors*. https://www.hse.gov.uk/ventilation/using-co2-monitors.htm

MMA3001 dataset - MSI occupancy and environmental. (n.d.). [Data set]. Google Drive. https://drive.google.com/drive/folders/1sB4UyZc4c6h67iKfmWcCm0Kv3DSjnCb2

NumPy. (n.d.). *numpy.linalg.lstsq*. https://numpy.org/doc/stable/reference/generated/numpy.linalg.lstsq.html

SciPy. (n.d.). *CubicSpline*. https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.html